

Docker



Índice

1 Introducción a Docker	3
1.1 ¿Qué es Docker y por qué debería usarlo?	3
1.2 Instalación y Configuración	4
1.3 Ejercicios del Módulo 1	7
1.4 Recursos y Referencias	8
2 Los Fundamentos de Docker	9
2.1 Imágenes y Contenedores	9
2.2 Creando tus Propias Imágenes con Dockerfile	10
2.3 Persistencia de Datos con Volúmenes	12
2.4 Redes en Docker	13
2.5 Ejercicios del Módulo 2	14
2.6 Recursos y Referencias	16
3 Orquestación con Docker Compose	17
3.1 Introducción a Docker Compose	17
3.2 Creando un Entorno de Desarrollo Completo	20
3.3 Ejercicios del Módulo 3	23
3.4 Recursos y Referencias	25
4 Docker en el Flujo de Trabajo de Desarrollo	26
4.1 Optimización de Imágenes y Builds	26
4.2 Integración con Herramientas de Desarrollo	28
4.3 Ejercicios del Módulo 4	30
4.4 Recursos y Referencias	32
5 Docker en Producción y Despliegue	33
5.1 Seguridad en Docker	33
5.2 Estrategias de Despliegue	34
5.3 CI/CD (Integración y Despliegue Continuo) con Docker	35
5.4 Ejercicios del Módulo 5	37
5.5 Recursos y Referencias	38
6 Anexos	40
6.1 Apéndice A: Resumen de Comandos de Docker y Docker Compose	40
6.2 Apéndice B: Solución de Problemas Comunes	43
6.3 Apéndice C: Recursos Adicionales y Comunidad	45

1 Introducción a Docker

¡Bienvenido al inicio de tu viaje con Docker! Este primer módulo está diseñado para sentar unas bases sólidas. Entenderás qué es exactamente Docker, por qué se ha convertido en una herramienta tan indispensable para los desarrolladores web modernos y cómo dar tus primeros pasos instalándolo y ejecutando tu primer contenedor.

1.1 ¿Qué es Docker y por qué debería usarlo?

1.1.1 Introducción a los contenedores

Imagina que eres un desarrollador y acabas de terminar una increíble aplicación web en tu portátil. Usa Node.js, una base de datos PostgreSQL y Redis para la caché. Funciona a la perfección. Ahora, se la pasas a un compañero para que la pruebe, o intentas desplegarla en un servidor. De repente, empiezan los problemas: "Necesitas una versión diferente de Node.js", "La base de datos no se conecta", "Falta una librería del sistema"... Este es el clásico problema de **"en mi máquina funciona"**.

Aquí es donde entran en juego los **contenedores**.

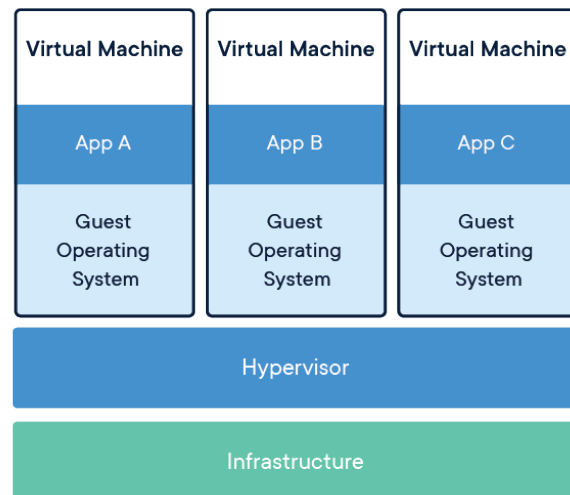
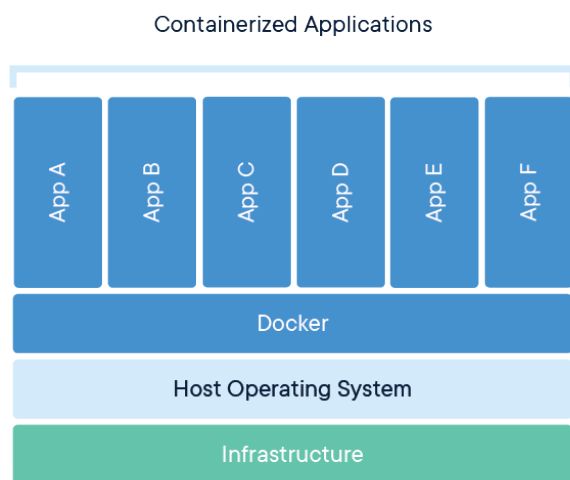
Un contenedor es un paquete que incluye una aplicación y todas sus dependencias (código, librerías, herramientas del sistema, etc.) en un solo lugar. Este paquete es ligero, portátil y se ejecuta de manera aislada del resto del sistema.

Piensa en un contenedor como una caja de LEGO perfectamente sellada: dentro tienes todas las piezas y las instrucciones para construir tu castillo (tu aplicación), y no importa dónde lleves esa caja (otro ordenador, un servidor en la nube), siempre podrás construir el mismo castillo exactamente de la misma manera.

1.1.2 Diferencias clave entre contenedores y máquinas virtuales

Es muy común confundir los contenedores con las Máquinas Virtuales (VMs), ya que ambos proporcionan entornos aislados. Sin embargo, su arquitectura es fundamentalmente diferente.

- **Máquinas Virtuales (VMs):** Una VM emula un ordenador completo, incluyendo su propio sistema operativo (SO). Sobre el hardware físico de tu máquina, tienes un "Hipervisor" (como VirtualBox o VMware) que gestiona varias VMs. Cada VM tiene su propio SO, lo que consume una cantidad significativa de recursos (CPU, RAM, espacio en disco).
- **Contenedores:** Los contenedores, en cambio, comparten el kernel del sistema operativo de la máquina anfitriona. No emulan hardware ni necesitan un SO completo. Docker es el motor que gestiona estos contenedores. Esto los hace increíblemente ligeros y rápidos. Puedes levantar un contenedor en milisegundos, mientras que una VM puede tardar minutos.



Característica	Máquinas Virtuales	Contenedores
Aislamiento	A nivel de Hardware	A nivel de Proceso
Recursos	Consumo alto (GBs)	Consumo bajo (MBs)
Arranque	Lento (minutos)	Rápido (segundos)
Portabilidad	Moderada	Alta
Contenido	SO completo + App	Solo la App y sus dependencias

1.1.3 Beneficios de Docker en el desarrollo web

1. **Entornos de desarrollo consistentes:** Garantiza que todos los miembros del equipo (y los servidores de producción) ejecuten la aplicación en un entorno idéntico. Adiós al "en mi máquina funciona".
2. **Portabilidad:** Una aplicación "dockerizada" se ejecuta en cualquier lugar donde Docker esté instalado: en el portátil de un compañero, en un servidor de la empresa o en la nube (AWS, Google Cloud, Azure).
3. **Aislamiento:** Puedes ejecutar diferentes proyectos en tu máquina con diferentes versiones de software (ej. PHP 7.4 y PHP 8.1) sin que entren en conflicto.
4. **Despliegue rápido y sencillo:** Los contenedores simplifican enormemente el proceso de llevar una aplicación a producción.
5. **Microservicios:** Docker es la tecnología base para construir arquitecturas de microservicios, donde cada servicio se ejecuta en su propio contenedor.

1.2 Instalación y Configuración

Es hora de pasar a la acción. En este capítulo instalaremos Docker y ejecutaremos nuestro primer contenedor.

1.2.1 Instalación de Docker Desktop

Docker ofrece una aplicación de escritorio llamada **Docker Desktop** que incluye el motor de Docker, la herramienta de línea de comandos (CLI) y una interfaz gráfica. Es la forma más sencilla de empezar.

- **Para Windows:**

1. Ve a la [página oficial de Docker Desktop](#).
2. Asegúrate de que tu sistema cumple los requisitos (normalmente Windows 10/11 de 64 bits y la activación de WSL 2, el instalador te guiará).
3. Descarga y ejecuta el instalador. Sigue las instrucciones y reinicia tu equipo si es necesario.

- **Para macOS:**

1. Ve a la [página oficial de Docker Desktop](#).
2. Elige la versión correcta para tu Mac (chip Intel o Apple Silicon).
3. Descarga el archivo .dmg, ábrelo y arrastra el icono de Docker a tu carpeta de Aplicaciones.
4. Inicia Docker desde tus aplicaciones.

- **Para Linux:**

1. En Linux, la instalación se realiza generalmente a través de la línea de comandos. Docker Desktop también está disponible, pero es más común instalar solo el motor.
2. La mejor manera es seguir la guía oficial para tu distribución (Ubuntu, Debian, Fedora, etc.). Por ejemplo, para **Ubuntu**:

```
# Desinstalar versiones antiguas
sudo apt-get remove docker docker-engine docker.io containerd runc

# Configurar el repositorio de Docker
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/
keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalar Docker Engine
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
```

3. **Post-instalación (Importante):** Para poder ejecutar comandos de Docker sin sudo, añade tu usuario al grupo de Docker.

```
sudo groupadd docker # Puede que ya exista
sudo usermod -aG docker $USER
# Necesitarás cerrar sesión y volver a entrar para que el cambio
surta efecto.
newgrp docker
```

1.2.2 Un vistazo al Docker Hub

Docker Hub es un registro de imágenes de contenedores, algo así como GitHub pero para Docker. Contiene miles de imágenes oficiales y creadas por la comunidad que puedes usar como base para tus aplicaciones.

- **Imágenes oficiales:** Son imágenes mantenidas y verificadas por los proveedores de software (ej. node, python, postgres, nginx). Son la opción recomendada para empezar.
- **Imágenes de la comunidad:** Imágenes creadas y compartidas por otros usuarios.

Puedes explorar el hub en <https://hub.docker.com/>.

1.2.3 Tu primer comando: docker run hello-world

Para verificar que todo está instalado correctamente, abre tu terminal (o PowerShell en Windows) y ejecuta el siguiente comando:

```
docker run hello-world
```

Veamos qué sucede tras bambalinas cuando ejecutas este comando:

1. El cliente de Docker (docker) contacta al demonio de Docker (el motor que se ejecuta en segundo plano).
2. El demonio busca una imagen llamada hello-world en tu máquina local.
3. Como es la primera vez, no la encuentra.
4. El demonio se conecta a Docker Hub y busca la imagen hello-world oficial.
5. Descarga la imagen a tu máquina local. Esto se conoce como **docker pull**.
6. Una vez descargada, el demonio crea un nuevo **contenedor** a partir de esa imagen.
7. El contenedor se ejecuta, imprime un mensaje de bienvenida en tu terminal y luego se detiene.

Si todo ha ido bien, deberías ver un mensaje como este:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

¡Felicidades, has ejecutado tu primer contenedor!

1.2.4 Ejemplo práctico: Ejecutar un servidor web NGINX

Vamos a hacer algo un poco más útil. NGINX es un servidor web muy popular. Podemos levantarlo en un contenedor con un solo comando.

```
docker run --name mi-servidor-web -p 8080:80 -d nginx
```

Desglosemos este comando:

- `docker run`: El comando para crear y ejecutar un contenedor.
- `--name mi-servidor-web`: Le damos un nombre fácil de recordar a nuestro contenedor (`mi-servidor-web`). Si no lo hacemos, Docker le asignará uno aleatorio.
- `-p 8080:80`: Esto es crucial. Mapea el puerto 8080 de tu máquina (el anfitrión) al puerto 80 del contenedor. NGINX escucha en el puerto 80 por defecto. Sin esto, no podríamos acceder al servidor desde nuestro navegador.
- `-d`: Ejecuta el contenedor en modo "detached" (desacoplado), es decir, en segundo plano. La terminal queda libre.
- `nginx`: El nombre de la imagen que queremos usar desde Docker Hub.

Ahora, abre tu navegador web y visita `http://localhost:8080`.

Deberías ver la página de bienvenida de NGINX. ¡Has desplegado un servidor web en segundos!

Para detener y eliminar el contenedor, puedes usar los siguientes comandos:

```
# Para detener el contenedor
docker stop mi-servidor-web

# Para eliminar el contenedor (solo se puede si está detenido)
docker rm mi-servidor-web
```

1.3 Ejercicios del Módulo 1

1. **Pregunta Teórica:** Explica con tus propias palabras la principal diferencia entre un contenedor y una máquina virtual en términos de uso de recursos.
2. **Comando de Verificación:** ¿Qué comando usarías para comprobar si tu instalación de Docker funciona correctamente y qué hace exactamente ese comando?
3. **Ejecutar un Contenedor:** Ejecuta un contenedor basado en la imagen oficial de redis (una base de datos en memoria). Dale el nombre `mi-cache`.
4. **Inspección de Contenedores:** ¿Qué comando usarías para ver todos los contenedores que se están ejecutando en tu sistema? ¿Y para ver todos los contenedores, incluyendo los que están detenidos?
5. **Limpieza:** Detén y elimina el contenedor `mi-cache` que creaste en el ejercicio 3.

1.3.1 Respuestas a los Ejercicios

1. **Respuesta:** La principal diferencia es que una máquina virtual incluye un sistema operativo completo para cada instancia, lo que consume una gran cantidad de RAM y espacio en disco. En cambio, los contenedores comparten el sistema operativo de la máquina anfitriona y solo empaquetan la aplicación y sus dependencias, siendo mucho más ligeros y rápidos.
2. **Respuesta:** El comando es `docker run hello-world`. Este comando busca la imagen `hello-world` localmente. Si no la encuentra, la descarga de Docker Hub, crea un nuevo contenedor a partir de ella, lo ejecuta (imprimiendo un mensaje) y luego lo detiene.
3. **Respuesta:**

```
docker run --name mi-cache -d redis
```

4. Respuesta:

- Para ver los contenedores en ejecución: `docker ps` (o `docker container ls`).
- Para ver todos los contenedores (incluidos los detenidos): `docker ps -a`.

5. Respuesta:

```
# Paso 1: Detener el contenedor
docker stop mi-cache

# Paso 2: Eliminar el contenedor
docker rm mi-cache
```

1.4 Recursos y Referencias

- **Documentación Oficial de Docker:** El mejor lugar para encontrar información detallada y precisa.
 - [Get started with Docker](#)
 - [Guías de instalación oficiales](#)
- **Docker Hub:** Explora miles de imágenes listas para usar.
 - [Docker Hub](#)
- **Play with Docker:** Un entorno de Docker online y gratuito para experimentar sin instalar nada.
 - [Play with Docker](#)

2 Los Fundamentos de Docker

Ahora que ya tienes una idea general de lo que es Docker y has ejecutado tus primeros contenedores, es hora de profundizar en los pilares de su funcionamiento. En este módulo, aprenderás a manejar los dos conceptos más importantes, **imágenes** y **contenedores**, y descubrirás cómo construir tus propias imágenes personalizadas con **Dockerfiles**. Además, abordaremos cómo hacer que los datos de tus aplicaciones sobrevivan y cómo conectar diferentes contenedores entre sí.

2.1 Imágenes y Contenedores

2.1.1 Imágenes: La Plantilla de tu Aplicación

Una imagen de Docker es una plantilla inmutable (no se puede cambiar) que contiene todo lo necesario para ejecutar una aplicación: el código, un entorno de ejecución (como Node.js o Python), librerías, variables de entorno y archivos de configuración.

Las imágenes se construyen a partir de una serie de capas apiladas una sobre otra. Cada capa representa una instrucción del Dockerfile (lo veremos en el próximo capítulo). Esta estructura por capas es muy eficiente, ya que permite que diferentes imágenes compartan las capas comunes, ahorrando espacio en disco y acelerando las compilaciones y descargas.

Comandos clave para Imágenes:

- `docker images`: Lista todas las imágenes que tienes descargadas en tu máquina.
- `docker pull <nombre_imagen>:<tag>`: Descarga una imagen (y su versión o "tag") desde un registro como Docker Hub. Si no especificas un tag (ej. `:latest`, `:18.04`), por defecto se usa `latest`.
 - **Ejemplo:** `docker pull ubuntu:22.04`
- `docker rmi <id_imagen_o_nombre>`: Elimina una imagen de tu máquina. Solo podrás eliminar una imagen si no hay ningún contenedor (ni siquiera detenido) que la esté usando.
 - **Ejemplo:** `docker rmi ubuntu:22.04`

2.1.2 Contenedores: Instancias en Ejecución de tus Imágenes

Si una imagen es la receta, un contenedor es el pastel ya horneado y listo para comer. Es una instancia viva y en ejecución de una imagen. Puedes crear, iniciar, detener, mover y eliminar contenedores. Cada contenedor es un entorno aislado y seguro que tiene su propio sistema de archivos, procesos y red.

Puedes ejecutar múltiples contenedores a partir de la misma imagen. Cada uno funcionará de forma independiente.

Comandos esenciales para Contenedores:

- `docker run <nombre_imagen>`: Crea y arranca un nuevo contenedor a partir de una imagen. Ya vimos algunas de sus opciones (`--name`, `-p`, `-d`) en el módulo anterior.
- `docker ps`: Lista los contenedores que están **actualmente en ejecución**.
- `docker ps -a`: Lista **todos** los contenedores, incluyendo los que están detenidos.
- `docker start <id_contenedor_o_nombre>`: Inicia un contenedor que estaba detenido.
- `docker stop <id_contenedor_o_nombre>`: Detiene la ejecución de un contenedor de forma elegante (envía una señal `SIGTERM`).

- `docker kill <id_contenedor_o_nombre>`: Detiene la ejecución de un contenedor de forma abrupta (envía una señal SIGKILL).
 - `docker rm <id_contenedor_o_nombre>`: Elimina un contenedor que está detenido. Para eliminar un contenedor en ejecución, debes forzarlo con `docker rm -f <id>`.
 - `docker logs <id_contenedor_o_nombre>`: Muestra los logs (salida estándar) de un contenedor. Muy útil para depurar.
 - **Ejemplo:** `docker logs mi-servidor-web`
 - Usa la opción `-f` para seguir los logs en tiempo real: `docker logs -f mi-servidor-web`
 - `docker exec -it <id_contenedor_o_nombre> <comando>`: Ejecuta un comando dentro de un contenedor que ya está en marcha. La opción `-it` (interactivo + TTY) te permite abrir una terminal dentro del contenedor.
 - **Ejemplo:** `docker exec -it mi-servidor-web bash` (Esto abre una sesión de terminal bash dentro del contenedor de NGINX).
-

2.2 Creando tus Propias Imágenes con Dockerfile

Hasta ahora hemos usado imágenes pre-hechas de Docker Hub. Pero el verdadero poder de Docker reside en poder empaquetar **nuestras propias aplicaciones**. Para ello, usamos un archivo de texto llamado Dockerfile.

Un Dockerfile es una receta con instrucciones paso a paso que Docker sigue para construir una imagen personalizada.

2.2.1 Estructura y Comandos Clave de un Dockerfile

- `FROM <imagen>:<tag>`: Siempre es la primera instrucción. Define la imagen base sobre la que construiremos la nuestra. (Ej: `FROM node:18-alpine`).
- `WORKDIR /ruta/en/el/contenedor`: Establece el directorio de trabajo para las siguientes instrucciones (`RUN`, `COPY`, `CMD`, etc.). Si no existe, lo crea. Es como hacer `cd` en la terminal.
- `COPY <origen_local> <destino_en_contenedor>`: Copia archivos o directorios desde tu máquina local al sistema de archivos del contenedor.
- `RUN <comando>`: Ejecuta un comando durante el proceso de construcción de la imagen. Se usa para instalar paquetes, compilar código, etc. Cada `RUN` crea una nueva capa en la imagen.
- `EXPOSE <puerto>`: Informa a Docker que el contenedor escuchará en un puerto de red específico en tiempo de ejecución. Es una instrucción meramente informativa, no publica el puerto.
- `CMD ["ejecutable", "param1", "param2"]`: Define el comando por defecto que se ejecutará cuando se inicie un contenedor a partir de esta imagen. Solo puede haber un `CMD` por Dockerfile.

2.2.2 Práctica: Creando una Imagen para una App Web Node.js

Vamos a crear una aplicación web muy simple con Express.js y a "dockerizarla".

1. **Crea una carpeta para tu proyecto:** `mkdir mi-app-web && cd mi-app-web`
2. **Crea el archivo `server.js`:**

```
// server.js
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('¡Hola Mundo desde un contenedor Docker!');
});

app.listen(PORT, () => {
  console.log(`Servidor escuchando en el puerto ${PORT}`);
});
```

3. Crea el archivo package.json:

```
{
  "name": "mi-app-web",
  "version": "1.0.0",
  "description": "Una simple app web con Node.js",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

4. Ahora, crea el Dockerfile (¡sin extensión de archivo!):

```
# Dockerfile

# 1. Usar una imagen base oficial de Node.js.
# 'alpine' es una versión muy ligera de Linux, ideal para producción.
FROM node:18-alpine

# 2. Establecer el directorio de trabajo dentro del contenedor.
WORKDIR /app

# 3. Copiar el package.json y package-lock.json (si existe).
# Se copian por separado para aprovechar el cache de capas de Docker.
COPY package*.json ./

# 4. Instalar las dependencias de la aplicación.
RUN npm install

# 5. Copiar el resto de los archivos de la aplicación al directorio de trabajo.
COPY . .

# 6. Exponer el puerto en el que la aplicación se ejecuta.
EXPOSE 3000

# 7. Definir el comando para ejecutar la aplicación.
CMD [ "npm", "start" ]
```

5. **Construye la imagen:** Abre tu terminal en la carpeta del proyecto y ejecuta:

```
# El comando 'build' construye la imagen.
# '-t' es para 'tag', le damos un nombre y una versión.
# '.' indica que el Dockerfile está en el directorio actual.
docker build -t mi-app-web:1.0 .
```

6. **Ejecuta un contenedor con tu nueva imagen:**

```
docker run --name mi-aplicacion -p 8080:3000 -d mi-app-web:1.0
```

- Mapeamos el puerto 8080 de nuestra máquina al puerto 3000 expuesto por el contenedor.

¡Listo! Abre tu navegador y visita <http://localhost:8080>. Verás el mensaje "¡Hola Mundo desde un contenedor Docker!". Has creado y ejecutado tu primera aplicación dockerizada.

2.3 Persistencia de Datos con Volúmenes

Cuando un contenedor es eliminado, todos los datos creados dentro de él se pierden. Esto es un problema para aplicaciones que necesitan guardar información, como las bases de datos.

Los **volúmenes** son el mecanismo preferido por Docker para persistir datos. Son directorios especiales gestionados por Docker que se montan en los contenedores. La clave es que **el ciclo de vida de un volumen es independiente del ciclo de vida de un contenedor**.

Ventajas de los volúmenes:

- Puedes crear, eliminar y listar volúmenes con comandos de Docker (`docker volume create`, etc.).
- Son más seguros y eficientes que hacer "bind mounts" (montar un directorio de tu máquina directamente).
- Funcionan igual en todos los sistemas operativos (Windows, macOS, Linux).

2.3.1 Práctica: Configurando una Base de Datos PostgreSQL con un Volumen

Vamos a crear un contenedor de PostgreSQL y nos aseguraremos de que sus datos no se pierdan si eliminamos el contenedor.

1. Crea un volumen gestionado por Docker:

```
docker volume create datos-postgres
```

2. Ejecuta el contenedor de PostgreSQL usando el volumen:

```
docker run --name mi-postgres-db \
-e POSTGRES_PASSWORD=mi_clave_secreta \
-v datos-postgres:/var/lib/postgresql/data \
-p 5432:5432 \
-d postgres
```

Desglose del comando:

- `-e POSTGRES_PASSWORD=...`: Establecemos una variable de entorno requerida por la imagen de postgres para fijar la contraseña del superusuario.
- `-v datos-postgres:/var/lib/postgresql/data`: Esta es la parte importante. `-v` monta un volumen.
 - `datos-postgres`: Es el nombre del volumen que creamos.
 - `/var/lib/postgresql/data`: Es la ruta **dentro del contenedor** donde PostgreSQL guarda todos sus datos.
 - Conectamos nuestro volumen a esa ruta. Ahora, todos los datos de la BD se escribirán en el volumen `datos-postgres`, fuera del contenedor.
- `-p 5432:5432`: Mapeamos el puerto para poder conectar a la BD desde nuestra máquina.

Ahora puedes detener (`docker stop mi-postgres-db`) y eliminar (`docker rm mi-postgres-db`) el contenedor. Luego, si vuelves a ejecutar el mismo comando `docker run`, el nuevo contenedor se conectará al volumen existente y ¡toda tu información seguirá ahí!

2.4 Redes en Docker

Por defecto, los contenedores que ejecutas están aislados. Si quieres que tu contenedor de backend (la app de Node.js) se comuniquen con tu contenedor de base de datos (PostgreSQL), necesitas conectarlos a una red común.

Docker permite crear redes virtuales para este propósito. Los contenedores en la misma red pueden comunicarse entre sí usando sus nombres como si fueran nombres de host (DNS).

2.4.1 Práctica: Conectando nuestra App Node.js con la Base de Datos

1. Crea una red personalizada:

```
docker network create mi-red-app
```

2. Detén y elimina los contenedores anteriores (si aún existen):

```
docker stop mi-aplicacion mi-postgres-db  
docker rm mi-aplicacion mi-postgres-db
```

3. Ejecuta el contenedor de la base de datos, conectándolo a la nueva red:

```
docker run --name mi-postgres-db \  
  --network mi-red-app \  
  -e POSTGRES_PASSWORD=mi_clave_secreta \  
  -v datos-postgres:/var/lib/postgresql/data \  
  -d postgres
```

- La única diferencia es `--network mi-red-app`. No necesitamos exponer el puerto (`-p`) si solo la app se va a conectar.

4. Ejecuta el contenedor de la aplicación, conectándolo también a la red:

```
docker run --name mi-aplicacion \  
  --network mi-red-app \  
  -p 8080:3000 \  
  -d mi-app-web:1.0
```

- Ahora, desde el código de la aplicación Node.js (que en un caso real tendrías que modificar), podrías conectarte a la base de datos usando `mi-postgres-db` como el nombre del host. Por ejemplo, la cadena de conexión sería `postgresql://postgres:mi_clave_secreta@mi-postgres-db:5432/nombre_db`. Docker se encarga de la resolución de nombres.

2.5 Ejercicios del Módulo 2

1. **Construir una Imagen:** Crea un Dockerfile para una aplicación web sencilla escrita en Python con Flask. La app debe mostrar "Hola desde Python!".
2. **Gestión de Contenedores:** Construye la imagen del ejercicio 1 y llámala `python-app:1.0`. Luego, ejecuta un contenedor llamado `mi-python-app` que sea accesible en el puerto 5000 de tu máquina. Finalmente, mira sus logs.

3. **Uso de Volúmenes:** Ejecuta un contenedor de `mysql:8.0`. Asegúrate de que sus datos persistan en un volumen llamado `datos-mysql`. Debes configurar la contraseña de root a través de una variable de entorno (`MYSQL_ROOT_PASSWORD`).
4. **Redes:** Crea una red llamada `test-net`. Ejecuta un contenedor `nginx` llamado `web-server` y un contenedor `ubuntu` llamado `client` en esa misma red. Accede a la terminal del contenedor `client` y usa `curl http://web-server` para verificar que puedes comunicarte con NGINX usando su nombre.

2.5.1 Respuestas a los Ejercicios

1. Respuesta (app.py y Dockerfile):

- Crea `app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hola desde Python!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- Crea `requirements.txt`:

```
Flask==2.2.2
```

- Crea `Dockerfile`:

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["flask", "run"]
```

2. Respuesta (Comandos):

```
# Construir la imagen
docker build -t python-app:1.0 .

# Ejecutar el contenedor
docker run --name mi-python-app -p 5000:5000 -d python-app:1.0

# Ver los logs
docker logs mi-python-app
```

3. Respuesta (Comando):

```
# Crear el volumen (opcional, se puede crear al vuelo)
docker volume create datos-mysql

# Ejecutar el contenedor de MySQL
docker run --name mi-mysql-db \
  -v datos-mysql:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=mi_clave \
  -d mysql:8.0
```

4. Respuesta (Comandos):

```
# Crear la red
docker network create test-net

# Ejecutar el servidor NGINX en la red
docker run --name web-server --network test-net -d nginx

# Ejecutar un cliente Ubuntu, acceder a su terminal y hacer curl
# La opción --rm elimina el contenedor cuando salimos de la terminal
docker run --rm -it --network test-net --name client ubuntu bash

# Una vez dentro del contenedor 'client':
# apt-get update && apt-get install -y curl
# curl http://web-server
# exit
```

2.6 Recursos y Referencias

- **Dockerfile reference:** La guía oficial para cada instrucción del Dockerfile.
 - <https://docs.docker.com/engine/reference/builder/>
- **Best practices for writing Dockerfiles:** Consejos para crear imágenes optimizadas y seguras.
 - https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- **Documentación sobre Volúmenes:** Todo lo que necesitas saber sobre la persistencia de datos.
 - <https://docs.docker.com/storage/volumes/>
- **Conceptos de Redes en Docker:** Guía oficial sobre el funcionamiento de las redes.
 - <https://docs.docker.com/network/>

3 Orquestación con Docker Compose

Has aprendido a crear imágenes y a ejecutar contenedores individuales. Sin embargo, las aplicaciones web modernas rara vez consisten en un único componente. Normalmente tienes un backend, un frontend, una base de datos, un servicio de caché, etc. Gestionar todo esto con comandos `docker run` individuales, creando redes y volúmenes a mano, se vuelve tedioso y propenso a errores rápidamente.

Este módulo introduce **Docker Compose**, una herramienta para definir y ejecutar aplicaciones Docker multi-contenedor. Con un único archivo de configuración, podrás levantar y conectar todo tu entorno de desarrollo con un solo comando. □

3.1 Introducción a Docker Compose

3.1.1 ¿Qué es Docker Compose y para qué sirve?

Docker Compose es una herramienta de orquestación que utiliza un archivo de configuración en formato **YAML** (generalmente `docker-compose.yml`) para describir todos los servicios que componen tu aplicación. En este archivo defines los contenedores, las redes que los conectan, los volúmenes para la persistencia de datos y las configuraciones específicas de cada uno.

Los principales beneficios son:

- **Simplicidad:** Gestionas todo tu stack de aplicación desde un único archivo.
- **Reproducibilidad:** Cualquier persona con Docker puede clonar tu proyecto y levantar el mismo entorno con el comando `docker compose up`.
- **Eficiencia:** Automatiza los pasos de creación de redes, volúmenes y contenedores, y los conecta por ti.
- **Claridad:** El archivo `docker-compose.yml` sirve como una documentación clara de la arquitectura de tu aplicación.

3.1.2 Estructura de un archivo `docker-compose.yml`

Un archivo `docker-compose.yml` tiene una estructura jerárquica y muy legible. Veamos sus componentes principales:

```

# Versión del formato del archivo Compose. Es recomendable usar una versión
reciente.

# Aquí se definen todos los contenedores (servicios) de la aplicación.
services:

# Primer servicio: un backend de API. 'api' es el nombre que le damos.
api:
# Construye la imagen a partir de un Dockerfile en el directorio actual.
build: .
# Mapea el puerto 8000 del host al 8000 del contenedor.
ports:
- "8000:8000"
# Monta un volumen para la persistencia de datos.
volumes:
- datos_api:/app/data
# Variables de entorno para la configuración.
environment:
- DATABASE_URL=postgres://user:password@db:5432/mydb

# Segundo servicio: una base de datos PostgreSQL. 'db' es el nombre del servicio.
db:
# Usa una imagen pre-hecha de Docker Hub.
image: postgres:14-alpine
# Monta un volumen para que los datos de la BD no se pierdan.
volumes:
- datos_postgres:/var/lib/postgresql/data
# Variables de entorno para configurar la base de datos.
environment:
- POSTGRES_USER=user
- POSTGRES_PASSWORD=password
- POSTGRES_DB=mydb

# Aquí se definen los volúmenes que serán gestionados por Docker.
volumes:
datos_api:
datos_postgres:

# (Opcional) Definición de redes personalizadas.
# Si no se define, Compose crea una red por defecto para todos los servicios.
networks:
default:
driver: bridge

```

Claves importantes:

- **services:** Es el corazón del archivo. Cada clave bajo services (como api y db) es un contenedor individual.
- **Nombre del servicio (api, db):** Este nombre es crucial. Docker Compose lo usa como **nombre de host** dentro de la red que crea. Esto significa que el servicio api puede conectarse a la base de datos simplemente apuntando a db.
- **build vs image:** Usa build si necesitas construir una imagen desde un Dockerfile. Usa image si quieres usar una imagen ya existente de Docker Hub o de otro registro.
- **volumes:** La sección volumes al final del archivo declara los volúmenes "nombrados". Esto le dice a Docker que los gestione, desacoplado su ciclo de vida del de los contenedores.

3.1.3 Gestión de Secretos y Variables de Entorno (.env)

Hasta ahora hemos escrito las contraseñas y configuraciones directamente en el archivo `docker-compose.yml`. En un entorno real, esto es una **mala práctica de seguridad**.

Si subes tu código a un repositorio como GitHub, cualquiera podrá ver tus contraseñas. Para evitar esto, Docker Compose permite utilizar un archivo llamado `.env` para almacenar tus secretos y variables de configuración fuera del código.

3.1.3.1 Crear el archivo .env

Crea un archivo llamado `.env` en el mismo directorio donde está tu `docker-compose.yml`. Dentro, define tus variables con el formato `CLAVE=VALOR`:

```
# Archivo .env
# Configuración de la Base de Datos
DB_USER=mi_usuario_seguro
DB_PASSWORD=super_secreta_123
DB_NAME=mi_base_de_datos

# Configuración de Puertos
PORT_WEB=8080
```

3.1.3.2 Usar las variables en docker-compose.yml

Ahora, modifica tu archivo YAML para sustituir los valores fijos por la sintaxis `${NOMBRE_VARIABLE}`. Docker Compose leerá automáticamente el archivo `.env` y sustituirá los valores al ejecutar `up`.

```
version: '3.8'
services:
  database:
    image: postgres:14-alpine
    environment:
      # En lugar de escribir "mi_usuario_seguro", usamos la variable
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - datos_postgres:/var/lib/postgresql/data

  frontend:
    image: nginx:alpine
    ports:
      # También podemos parametrizar puertos
      - "${PORT_WEB}:80"
```

3.1.3.3 ¡Importante! El archivo .gitignore

El paso más importante es asegurarte de que **nunca** subes el archivo `.env` a tu repositorio de git.

Añade `.env` a tu archivo `.gitignore` inmediatamente:

```
# .gitignore
node_modules/
.DS_Store
.env <-- ¡Muy importante!
```

Consejo Profesional: Es común crear un archivo de ejemplo llamado `.env.example` con valores falsos o vacíos, y subirlo al repositorio. Así, tus compañeros sabrán qué variables necesitan configurar en sus propias máquinas para que el proyecto funcione.

3.2 Creando un Entorno de Desarrollo Completo

Vamos a aplicar lo aprendido para levantar un stack de desarrollo web típico: un **frontend** con NGINX, un **backend** con la API de Node.js que creamos en el módulo 2 y una base de datos PostgreSQL.

3.2.1 Estructura del Proyecto

Imagina que tu proyecto tiene esta estructura de carpetas:

```
mi_proyecto_completo/
├── backend/
│   ├── Dockerfile
│   ├── server.js
│   └── package.json
├── frontend/
│   ├── index.html
│   └── nginx.conf
└── docker-compose.yml
```

- **backend/**: Contiene la aplicación Node.js del Módulo 2.
- **frontend/**: Contiene un simple `index.html` y una configuración para NGINX.
- **docker-compose.yml**: El archivo que orquestrará todo.

3.2.2 Práctica: Levantando el Stack

1. Prepara los archivos:

- **backend/**: Usa los archivos `Dockerfile`, `server.js` y `package.json` del Módulo 2.
- **frontend/index.html**:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi App Frontend</title>
</head>
<body>
  <h1>Bienvenido al Frontend</h1>
  <p>Este sitio está siendo servido por un contenedor NGINX.</p>
</body>
</html>
```

- **frontend/nginx.conf**: Un archivo de configuración simple para NGINX.

```
server {  
    listen 80;  
    server_name localhost;  
  
    location / {  
        root /usr/share/nginx/html;  
        index index.html index.htm;  
    }  
}
```

2. Crea el archivo `docker-compose.yml` en la raíz del proyecto:

```

services:
  # Servicio del Backend (API)
  backend:
    build: ./backend
    container_name: mi_backend_api
    ports:
      - "3000:3000"
    volumes:
      # Montamos el código para hot-reloading en desarrollo
      - ./backend:/app
      # Excluimos node_modules para que use el del contenedor
      - /app/node_modules
    environment:
      - DATABASE_HOST=database
      - POSTGRES_USER=mi_usuario
      - POSTGRES_PASSWORD=mi_clave_secreta
      - POSTGRES_DB=mi_base_de_datos
    depends_on:
      - database

  # Servicio de la Base de Datos
  database:
    image: postgres:14-alpine
    container_name: mi_postgres_db
    ports:
      # Exponemos el puerto para poder conectar con un cliente de BD si es necesario
      - "5432:5432"
    volumes:
      - datos_db:/var/lib/postgresql/data
    environment:
      - POSTGRES_USER=mi_usuario
      - POSTGRES_PASSWORD=mi_clave_secreta
      - POSTGRES_DB=mi_base_de_datos

  # Servicio del Frontend (NGINX)
  frontend:
    image: nginx:alpine
    container_name: mi_frontend_web
    ports:
      - "8080:80"
    volumes:
      # Montamos nuestro HTML y la configuración de NGINX
      - ./frontend/index.html:/usr/share/nginx/html/index.html
      - ./frontend/nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - backend

volumes:
  datos_db:
    driver: local

```

- **depends_on:** Esta clave le dice a Compose el orden de arranque. frontend y backend no arrancarán hasta que database esté listo.
- **volumes en backend:** Fíjate en el montaje - ./backend:/app. Esto es genial para desarrollo: cualquier cambio que hagas en tu código local se reflejará instantáneamente dentro del contenedor (hot-reloading, si tu framework lo soporta).

3. Gestiona la aplicación con docker-compose

Abre una terminal en la raíz de `mi_proyecto_completo/` y usa estos comandos:

- **Levantar todo el stack:**

```
# El flag -d (detached) lo ejecuta en segundo plano.  
# --build reconstruye las imágenes si el Dockerfile ha cambiado.  
docker compose up -d --build
```

- **Ver los logs de todos los servicios (o de uno específico):**

```
docker compose logs -f  
docker compose logs -f backend
```

- **Detener y eliminar los contenedores, redes y volúmenes:**

```
# 'down' detiene y elimina los contenedores y la red.  
# '-v' adicionalmente elimina los volúmenes nombrados. ¡Úsalo con  
cuidado!  
docker compose down -v
```

- **Listar los servicios en ejecución:**

```
docker compose ps
```

Ahora puedes acceder a `http://localhost:8080` para ver tu frontend y la API estará disponible en `http://localhost:3000`.

3.3 Ejercicios del Módulo 3

1. **Stack de WordPress:** WordPress es una aplicación muy popular que necesita un servidor web (con PHP) y una base de datos MySQL. Crea un archivo `docker-compose.yml` que levante un servicio para WordPress y otro para MySQL.
 - Usa la imagen oficial `wordpress:latest`.
 - Usa la imagen oficial `mysql:5.7`.
 - WordPress necesita comunicarse con la base de datos. Investiga en Docker Hub qué **variables de entorno** necesita la imagen de WordPress para conectarse a la de MySQL.
 - Asegúrate de que los datos de MySQL persistan en un volumen nombrado.
 - Haz que el sitio de WordPress sea accesible en el puerto 8001 de tu máquina.
2. **Gestión del Ciclo de Vida:** Usando el `docker-compose.yml` que creaste para WordPress: a. Levanta la aplicación en segundo plano. b. Verifica que ambos contenedores (`wordpress` y `mysql`) están en ejecución. c. Detén la aplicación sin eliminar los contenedores. d. Vuelve a iniciarla. e. Finalmente, detén y elimina todos los recursos (contenedores y volúmenes).

3.3.1 Respuestas a los Ejercicios

1. Respuesta (docker-compose.yml para WordPress):

```
services:
  # Servicio de la base de datos MySQL
  db_wordpress:
    image: mysql:5.7
    container_name: wordpress_db
    # El volumen para persistir los datos de la base de datos
    volumes:
      - db_data:/var/lib/mysql
    # Variables de entorno para configurar la base de datos
    environment:
      MYSQL_ROOT_PASSWORD: clave_root_super_secreta
      MYSQL_DATABASE: wordpress_db
      MYSQL_USER: wordpress_user
      MYSQL_PASSWORD: clave_wordpress_muy_segura
    # Reiniciar siempre el contenedor si se cae
    restart: always

  # Servicio de WordPress
  wordpress:
    # 'depends_on' asegura que la base de datos se inicie primero
    depends_on:
      - db_wordpress
    image: wordpress:latest
    container_name: wordpress_site
    # Mapeo de puertos para acceder al sitio
    ports:
      - "8001:80"
    # Variables de entorno para que WordPress se conecte a la BD
    # Los valores deben coincidir con los del servicio db_wordpress
    environment:
      WORDPRESS_DB_HOST: db_wordpress:3306 # Nombre del servicio:puerto
      WORDPRESS_DB_USER: wordpress_user
      WORDPRESS_DB_PASSWORD: clave_wordpress_muy_segura
      WORDPRESS_DB_NAME: wordpress_db
    # Reiniciar siempre el contenedor si se cae
    restart: always

# Definición del volumen nombrado
volumes:
  db_data:
```

2. Respuesta (Comandos de gestión):


```
# a. Levantar la aplicación
docker compose up -d

# b. Verificar que están en ejecución
docker compose ps

# c. Detener la aplicación
docker compose stop

# d. Volver a iniciarla
docker compose start

# e. Detener y eliminar todo
docker compose down -v
```

3.4 Recursos y Referencias

- **Documentación Oficial de Docker Compose:** La fuente de verdad para todo lo relacionado con Compose.
 - [Visión General de Docker Compose](#)
 - [Referencia del archivo docker-compose.yml](#)
 - [Referencia de comandos de la CLI de Compose](#)
- **Awesome Compose:** Un repositorio de GitHub con ejemplos de archivos docker-compose.yml para diferentes stacks tecnológicos. Muy útil para inspirarse.
 - [GitHub - Awesome Compose](#)

4 Docker en el Flujo de Trabajo de Desarrollo

Has dominado los fundamentos y sabes cómo orquestar una aplicación completa con Docker Compose. Ahora es el momento de refinar tus habilidades. Este módulo se centra en dos aspectos cruciales: cómo construir imágenes que sean pequeñas, rápidas y seguras, y cómo integrar Docker de manera fluida en tu entorno de desarrollo para maximizar tu productividad.

Aprenderás técnicas profesionales para optimizar tus builds y para que Docker, en lugar de ser un paso más, se convierta en una extensión natural de tu editor de código.

4.1 Optimización de Imágenes y Builds

El tamaño y la eficiencia de tus imágenes de Docker son importantes. Imágenes más pequeñas se descargan más rápido, ocupan menos espacio en disco y en los registros, y a menudo presentan una menor superficie de ataque a nivel de seguridad.

4.1.1 Uso de `.dockerignore`

De la misma manera que usas un archivo `.gitignore` para excluir archivos de tu repositorio de Git, puedes usar un archivo `.dockerignore` para excluir archivos y carpetas de ser enviados al demonio de Docker durante el proceso de construcción (`docker build`).

Cuando ejecutas `docker build`, el primer paso es que el cliente de Docker empaqueta el directorio actual (el "contexto de build") y lo envía al demonio. Si en ese directorio tienes carpetas como `node_modules`, `logs`, o archivos temporales, estarás enviando megabytes (o gigabytes) de datos innecesarios, ralentizando la construcción.

Ejemplo de un archivo `.dockerignore` para un proyecto Node.js:

```
# .dockerignore

# Logs y dependencias locales
logs
*.log
node_modules
npm-debug.log

# Archivos del sistema operativo y del editor
.DS_Store
.vscode/
```

Coloca este archivo en la raíz de tu contexto de build, al mismo nivel que tu `Dockerfile`.

4.1.2 Técnicas para Reducir el Tamaño de las Imágenes

1. **Elige la Imagen Base Correcta:** No todas las imágenes base son iguales. Una imagen `ubuntu` completa puede pesar cientos de MB, mientras que una imagen `alpine` (una distribución de Linux minimalista) puede pesar solo ~5 MB. Para muchas aplicaciones web, las variantes `-alpine` o `-slim` de las imágenes oficiales (ej. `node:18-alpine`, `python:3.9-slim`) son una excelente opción para producción.
2. **Combina Comandos RUN:** Cada instrucción `RUN` en un `Dockerfile` crea una nueva capa en la imagen. Puedes reducir el número de capas y el tamaño final combinando comandos con el

operador `&&`. Esto es especialmente útil para limpiar cachés de paquetes en la misma capa en que los instalas.

Ineficiente (2 capas):

```
RUN apt-get update
RUN apt-get install -y curl
```

Eficiente (1 capa y más pequeño):

```
RUN apt-get update && apt-get install -y curl \
    && rm -rf /var/lib/apt/lists/*
```

Al eliminar la caché (`/var/lib/apt/lists/*`) en la misma capa, te aseguras de que no ocupe espacio en la imagen final.

4.1.3 Builds Multi-Etapa (Multi-stage builds)

Esta es una de las técnicas de optimización más potentes. Un "build multi-etapa" te permite usar varias imágenes FROM en un mismo Dockerfile. Puedes usar una imagen más grande con todas las herramientas de compilación y dependencias de desarrollo (como compiladores, build-essentials, etc.) en una primera etapa, y luego copiar **únicamente los artefactos resultantes** (ej. un ejecutable compilado, la carpeta dist de una app frontend) a una imagen de producción final mucho más pequeña y limpia.

Práctica: Optimizando nuestra App Node.js

Imaginemos que nuestra app Node.js es más compleja y necesita un paso de "build" (por ejemplo, si usamos TypeScript).

Nuestro Dockerfile del Módulo 2 podría verse así con un build multi-etapa:

```
# Dockerfile optimizado

# --- Etapa 1: Build ---
# Usamos la imagen completa de Node para instalar dependencias y construir la app.
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
# Instalamos todas las dependencias, incluyendo las de desarrollo (devDependencies)
RUN npm install
COPY . .
# Este comando crearía una carpeta 'dist' con el código optimizado
RUN npm run build

# --- Etapa 2: Producción ---
# Empezamos desde cero con una imagen ligera de Alpine.
FROM node:18-alpine
WORKDIR /app
# Solo copiamos las dependencias de producción (no las de desarrollo)
COPY --from=builder /app/package*.json ./
RUN npm install --production
# Copiamos ÚNICAMENTE el código construido de la etapa anterior.
COPY --from=builder /app/dist ./dist
EXPOSE 3000
CMD [ "node", "dist/server.js" ]
```

Beneficios:

- **Imagen final diminuta:** La imagen final no contiene devDependencies, compiladores, ni ningún otro archivo intermedio. Solo el código necesario para ejecutar la aplicación.
- **Mayor seguridad:** Al no incluir herramientas de desarrollo en la imagen de producción, se reduce la superficie de ataque.

4.2 Integración con Herramientas de Desarrollo

El objetivo es que trabajar con Docker sea tan cómodo como trabajar sin él.

4.2.1 Uso de Docker con Visual Studio Code

VS Code, a través de su extensión oficial de **Docker**, ofrece una integración fantástica.

1. **Instala la extensión:** Busca "Docker" en el panel de extensiones de VS Code e instálala.
2. **Explorador de Docker:** Una vez instalada, tendrás un nuevo icono de una ballena en la barra lateral. Desde ahí puedes:
 - Ver y gestionar tus contenedores (iniciar, detener, ver logs, abrir una terminal dentro de ellos).
 - Ver tus imágenes, volúmenes y redes.
 - Hacer clic derecho en un Dockerfile o docker-compose.yml para construir o levantar los servicios directamente.

4.2.2 "Hot Reloading" para un Desarrollo Ágil

El ciclo de "editar código -> reconstruir imagen -> reiniciar contenedor" es lento y mata la productividad. Para el desarrollo, queremos que los cambios en nuestro código se reflejen al instante en la aplicación en ejecución. Esto se logra con **bind mounts**, que ya vimos brevemente en el Módulo 3.

Un "bind mount" sincroniza una carpeta de tu máquina local (host) con una carpeta dentro del contenedor.

Ejemplo con docker-compose.yml para nuestra app de Node.js:

```
# docker-compose.yml (sección del backend)
services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    # La magia del hot-reloading está aquí:
    volumes:
      # Monta el código fuente local en el workdir del contenedor.
      - ./backend:/app
      # Para evitar conflictos, declaramos un volumen anónimo para node_modules.
      # Esto le dice a Docker: "usa el node_modules que se instaló DENTRO de la
imagen,
      # no el que pueda existir en mi máquina local".
      - /app/node_modules
```

Con esta configuración, si tienes una herramienta como nodemon en tu proyecto Node.js, detectará los cambios de archivo que haces en tu editor local y reiniciará el servidor automáticamente dentro del contenedor. ¡No necesitas reconstruir la imagen para cada cambio!

4.2.3 Depuración de Aplicaciones dentro de un Contenedor

Depurar una aplicación que corre dentro de un contenedor puede parecer complicado, pero es totalmente factible. La idea general es:

1. **Exponer el puerto de depuración:** La mayoría de los entornos de ejecución (Node.js, Python, Java) pueden iniciarse en un modo de depuración que escucha en un puerto específico. Debes exponer ese puerto en tu docker-compose.yml.
2. **Iniciar la aplicación en modo de depuración:** Debes modificar el CMD o el comando de inicio para que incluya los flags de depuración.
3. **Configurar el depurador del editor:** En VS Code, configuras un archivo launch.json para que se "adjunte" al proceso que se ejecuta dentro del contenedor a través del puerto de depuración que expusiste.

Ejemplo simplificado para Node.js:

- docker-compose.yml:

```
# ...
ports:
  - "3000:3000" # Puerto de la app
  - "9229:9229" # Puerto de depuración de Node.js
# El comando ahora incluye el flag --inspect
command: node --inspect=0.0.0.0:9229 server.js
```

- .vscode/launch.json:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Docker: Attach to Node",
      "type": "node",
      "request": "attach",
      "port": 9229,
      "address": "localhost",
      "localRoot": "${workspaceFolder}/backend",
      "remoteRoot": "/app",
      "protocol": "inspector"
    }
  ]
}

```

Ahora puedes poner breakpoints en tu código en VS Code y el depurador se detendrá en ellos, aunque el código se esté ejecutando dentro de un contenedor.

4.3 Ejercicios del Módulo 4

1. **Optimización Multi-Etapa:** Toma el Dockerfile de la aplicación Python/Flask del ejercicio del Módulo 2 y conviértelo en un build multi-etapa. La primera etapa debe instalar las dependencias, y la segunda debe ser una imagen python:3.9-slim que solo contenga el código y las dependencias ya instaladas, sin el compilador.
2. **.dockerignore:** Crea un archivo .dockerignore apropiado para un proyecto Python. Debe ignorar los archivos de caché de Python (__pycache__), los entornos virtuales (ej. venv, .venv) y los archivos del editor VS Code.
3. **Hot Reloading:** Modifica el docker-compose.yml del ejercicio de WordPress (Módulo 3) para que los cambios que hagas en una carpeta local wp-content (donde van los temas y plugins) se reflejen en el contenedor de WordPress sin necesidad de reconstruirlo.

4.3.1 Respuestas a los Ejercicios

1. **Respuesta (Dockerfile multi-etapa para Python):**

```

# --- Etapa 1: Builder ---
# Usamos una imagen completa para instalar dependencias.
FROM python:3.9 AS builder

WORKDIR /app
COPY requirements.txt .
# Instalamos las dependencias en una carpeta específica
RUN pip install --user -r requirements.txt

# --- Etapa 2: Producción ---
# Empezamos con una imagen ligera.
FROM python:3.9-slim
WORKDIR /app

# Copiamos el código de la aplicación
COPY app.py .
# Copiamos las dependencias ya instaladas desde la etapa 'builder'
COPY --from=builder /root/.local /root/.local

# Nos aseguramos de que el ejecutable de python encuentre los paquetes
ENV PATH=/root/.local/bin:$PATH

EXPOSE 5000
CMD ["flask", "run", "--host=0.0.0.0"]

```

2. Respuesta (.dockerignore para Python):

```

# .dockerignore

# Entornos virtuales
.env
venv
env/

# Caché de Python
__pycache__/
*.pyc
*.pyo
*.pyd

# Editor y sistema
.vscode/
.idea/
.DS_Store

```

3. **Respuesta (docker-compose.yml modificado para WordPress):** Primero, crea una carpeta wp-content en tu máquina local. Luego, modifica la sección wordpress del docker-compose.yml:

```
# ...
services:
  wordpress:
    # ... (resto de la configuración)
    ports:
      - "8001:80"
    # Añade esta sección de volúmenes:
    volumes:
      # Monta nuestra carpeta local en la carpeta wp-content del contenedor.
      - ./wp-content:/var/www/html/wp-content
# ...
```

Ahora, si añades un tema o un plugin a tu carpeta local wp-content, aparecerá inmediatamente en el panel de administración de tu sitio WordPress.

4.4 Recursos y Referencias

- **Best practices for writing Dockerfiles:** Guía oficial con excelentes consejos de optimización.
 - https://docs.docker.com/develop/develop-images/dockerfile_best-practices/
- **Multi-stage builds:** La documentación oficial de esta potente característica.
 - <https://docs.docker.com/build/building/multi-stage/>
- **Extensión de Docker para VS Code:**
 - [Marketplace de VS Code](#)
- **Depurar Node.js en un Contenedor Docker:** Un tutorial detallado de la documentación de VS Code.
 - <https://code.visualstudio.com/docs/containers/docker-nodejs>

5 Docker en Producción y Despliegue

¡Felicidades por llegar al último módulo! Ya sabes construir, ejecutar y orquestar aplicaciones con Docker. Es hora de dar el paso final: llevar tus aplicaciones a producción de una manera segura y eficiente.

En este módulo, abordaremos las mejores prácticas de seguridad para tus contenedores. Aprenderás a "subir" tus imágenes a un lugar centralizado llamado registro y cómo desplegarlas en un servidor. Finalmente, te introduciremos en el mundo de la Integración y Despliegue Continuo (CI/CD) para automatizar todo el proceso, convirtiéndote en un desarrollador verdaderamente moderno y eficiente.

□

5.1 Seguridad en Docker

Ejecutar aplicaciones en producción conlleva una gran responsabilidad. La seguridad no es una opción, es un requisito. Aunque los contenedores proporcionan un buen nivel de aislamiento, hay varias prácticas clave que debes seguir.

5.1.1 Ejecutar Contenedores como Usuario no-root

Por defecto, los procesos dentro de un contenedor se ejecutan como el usuario root. Esto es un riesgo de seguridad. Si un atacante lograra escapar del contenedor, tendría privilegios de root en la máquina anfitriona, lo que sería catastrófico.

La mejor práctica es crear un usuario sin privilegios dentro de tu Dockerfile y usarlo para ejecutar tu aplicación.

Práctica: Modificando el Dockerfile para usar un usuario no-root

```
FROM node:18-alpine

WORKDIR /app

# Crear un usuario y grupo específicos para la aplicación
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copiar archivos y cambiar el propietario a nuestro nuevo usuario
COPY --chown=appuser:appgroup . .

# Cambiar al nuevo usuario. Las siguientes instrucciones se ejecutarán como 'appuser'
USER appuser

RUN npm install --production

EXPOSE 3000
CMD [ "node", "server.js" ]
```

5.1.2 Análisis de Vulnerabilidades en las Imágenes

Tus imágenes se basan en otras imágenes, las cuales a su vez se basan en otras... Esta cadena de dependencias puede introducir vulnerabilidades de seguridad conocidas (CVEs) en tu aplicación sin que te des cuenta.

Es crucial escanear tus imágenes en busca de estas vulnerabilidades. Docker incluye el comando `docker scan`, que utiliza el motor de Snyk para este propósito.

¿Cómo escanear una imagen?

Una vez que has construido tu imagen (por ejemplo, `mi-app-web:1.0`), puedes escanearla directamente desde la terminal:

```
docker scan mi-app-web:1.0
```

Este comando te dará un informe detallado de las vulnerabilidades encontradas, su severidad y, a menudo, cómo solucionarlas (por ejemplo, actualizando una librería o la imagen base).

5.1.3 Otras Consideraciones de Seguridad Clave

- **Usa imágenes base de confianza:** Utiliza siempre imágenes oficiales de Docker Hub (node, python, postgres, etc.) o imágenes de proveedores verificados.
- **Mantén las imágenes actualizadas:** Reconstruye tus imágenes periódicamente para incorporar los últimos parches de seguridad de las imágenes base.
- **Principio de mínimo privilegio:** No instales herramientas innecesarias en tu imagen de producción (como curl, git, o compiladores). Los builds multi-etapa son perfectos para esto.
- **Gestiona los secretos (secrets):** Nunca escribas contraseñas, tokens de API o claves en tus Dockerfile o docker-compose.yml. Usa variables de entorno cargadas desde archivos seguros o, en producción, los sistemas de gestión de secretos del orquestador (como Kubernetes Secrets o Docker Secrets).

5.2 Estrategias de Despliegue

Tienes una imagen segura y optimizada. Ahora, ¿cómo la llevas desde tu portátil a un servidor en la nube?

5.2.1 El Rol del Registro de Contenedores

Un **registro de contenedores** es un repositorio centralizado para almacenar y distribuir tus imágenes de Docker. Es como un GitHub, pero para imágenes. El flujo de trabajo es:

1. **Construyes** la imagen en tu máquina o en un sistema de CI/CD.
2. **Etiquetas (tag)** la imagen con el nombre del registro.
3. **Subes (push)** la imagen al registro.
4. **Descargas (pull)** la imagen en tu servidor de producción.

Registros Populares:

- **Docker Hub:** El registro por defecto. Ofrece repositorios públicos y privados.
- **GitHub Container Registry (GHCR):** Integrado perfectamente con GitHub y GitHub Actions.
- **Amazon ECR, Google Artifact Registry, Azure Container Registry:** Los registros de los principales proveedores cloud.

5.2.2 Flujo de Despliegue Básico

Vamos a ver cómo subir nuestra imagen `mi-app-web:1.0` a Docker Hub. (El proceso es muy similar para otros registros).

1. **Crea una cuenta** en [Docker Hub](#). Tu nombre de usuario será parte del nombre de la imagen.
2. **Inicia sesión (login) desde tu terminal:**

```
docker login
# Te pedirá tu usuario y contraseña (o un token de acceso).
```

3. **Etiqueta (tag) tu imagen:** Debes renombrar tu imagen para que siga el formato `<tu_usuario>/<nombre_repositorio>:<tag>`.

```
docker tag mi-app-web:1.0 tunombredeusuario/mi-app-web:1.0
```

4. **Sube (push) la imagen al registro:**

```
docker push tunombredeusuario/mi-app-web:1.0
```

5. **Despliega en el servidor de producción:** Ahora, en tu servidor, solo necesitas tener Docker instalado. Te conectarías por SSH y ejecutarías:

```
# Inicias sesión en el servidor (si es necesario para repositorios privados)
docker login

# Descargas y ejecutas la imagen en un solo comando
docker run -d -p 80:3000 tunombredeusuario/mi-app-web:1.0
```

5.2.3 El Siguiente Paso: Orquestadores

Ejecutar `docker run` manualmente funciona para aplicaciones simples. Pero, ¿qué pasa si la aplicación se cae? ¿Cómo la escalas para manejar más tráfico? Para esto existen los **orquestadores de contenedores** como **Kubernetes** o **Docker Swarm**.

Estas herramientas automatizan el despliegue, el escalado, el auto-reparado y la gestión de aplicaciones containerizadas a gran escala. Docker Compose es fantástico para el desarrollo, pero Kubernetes es el estándar de facto para la producción.

5.3 CI/CD (Integración y Despliegue Continuo) con Docker

La Integración Continua (CI) y el Despliegue Continuo (CD) son prácticas que automatizan las fases de construcción, prueba y despliegue de tu software. El objetivo es que cada vez que subes código nuevo a tu repositorio (ej. `git push`), un proceso automático se encargue de todo lo demás.

Docker es una pieza central en los flujos de CI/CD modernos.

5.3.1 ¿Qué es un Pipeline de CI/CD?

Un pipeline es una serie de pasos automatizados. Un pipeline típico con Docker podría ser:

1. **Trigger (Disparador):** Un desarrollador hace `git push` a la rama principal.
2. **Build (Construcción):** El servidor de CI (como GitHub Actions) clona el código, construye la imagen de Docker usando el `Dockerfile`.
3. **Test (Prueba):** (Opcional pero recomendado) Se ejecutan pruebas automatizadas contra la nueva imagen.
4. **Push (Subida):** Si las pruebas pasan, la imagen se etiqueta y se sube al registro de contenedores (Docker Hub, GHCR).
5. **Deploy (Despliegue):** El servidor de CI se conecta al servidor de producción y le ordena que descargue y ejecute la nueva versión de la imagen.

5.3.2 Práctica: Creando un Pipeline Básico con GitHub Actions

GitHub Actions es una herramienta de CI/CD integrada en GitHub. Los flujos de trabajo se definen en archivos YAML dentro de una carpeta `.github/workflows/` en tu repositorio.

Ejemplo: `.github/workflows/docker-publish.yml`

Este workflow se activará cada vez que se haga push a la rama `main`, construirá la imagen, y la subirá a GitHub Container Registry.

```

name: Docker Publish

# Se activa en cada push a la rama 'main'
on:
  push:
    branches: [ "main" ]

jobs:
  build_and_push:
    runs-on: ubuntu-latest # El trabajo se ejecuta en una máquina virtual de Ubuntu
    permissions:
      contents: read
      packages: write # Permisos para escribir en el registro de paquetes de GitHub

    steps:
      # 1. Clona el repositorio para tener acceso al código y al Dockerfile
      - name: Checkout repository
        uses: actions/checkout@v3

      # 2. Inicia sesión en GitHub Container Registry (GHCR)
      - name: Log in to the Container registry
        uses: docker/login-action@v2
        with:
          registry: ghcr.io
          username: ${github.actor} # El usuario que activó el workflow
          password: ${secrets.GITHUB_TOKEN} # Un token temporal que GitHub
genera

      # 3. Construye y sube la imagen de Docker
      - name: Build and push Docker image
        uses: docker/build-push-action@v4
        with:
          context: . # El contexto de build es la raíz del repo
          push: true # Le decimos que suba la imagen después de construirla
          # Etiqueta la imagen con el nombre del repo y el hash del commit
          tags: ghcr.io/${github.repository}:${github.sha}

```

Con este archivo en tu repositorio, cada push a main resultará en una nueva imagen de Docker, versionada y almacenada de forma segura, lista para ser desplegada.

5.4 Ejercicios del Módulo 5

1. **Seguridad:** Modifica el Dockerfile de la aplicación Python/Flask de los módulos anteriores para que se ejecute con un usuario no-root llamado flaskuser.
2. **Despliegue Manual:** Construye la imagen del ejercicio anterior, etiquétala para subirla a tu cuenta de Docker Hub y súbela. Verifica que aparece en tu perfil de Docker Hub.
3. **CI/CD:** Crea un repositorio en GitHub. Añade la aplicación Python/Flask (código y Dockerfile del ejercicio 1) y el archivo docker-publish.yml de ejemplo. Haz un push y comprueba en la pestaña "Actions" de tu repositorio si el workflow se ejecuta correctamente y si la imagen aparece en la sección "Packages" de tu perfil de GitHub.

5.4.1 Respuestas a los Ejercicios

1. **Respuesta (Dockerfile de Python seguro):**

```

# Usamos un build multi-etapa para optimizar
FROM python:3.9-slim AS builder
WORKDIR /usr/src/app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.9-slim
WORKDIR /usr/src/app
COPY --from=builder /usr/src/app .
COPY app.py .
# Creamos el usuario y grupo
RUN addgroup -S flaskgroup && adduser -S flaskuser -G flaskgroup
# Cambiamos el propietario de los archivos
RUN chown -R flaskuser:flaskgroup /usr/src/app
# Cambiamos al nuevo usuario
USER flaskuser
EXPOSE 5000
CMD ["flask", "run", "--host=0.0.0.0"]

```

2. Respuesta (Comandos de despliegue):

```

# Asume que la imagen se llama 'python-app-secure:1.0'
# Iniciar sesión
docker login

# Etiquetar la imagen
docker tag python-app-secure:1.0 tunombredeusuario/python-app:1.0

# Subir la imagen
docker push tunombredeusuario/python-app:1.0

```

3. Respuesta (Pasos para CI/CD):

1. Crea un nuevo repositorio en GitHub.
2. Localmente, crea la estructura de archivos: app.py, requirements.txt, Dockerfile (del ejercicio 1).
3. Crea la carpeta .github/workflows/ y dentro el archivo docker-publish.yml con el contenido del capítulo.
4. Haz git add ., git commit -m "Initial commit con app y workflow", y git push a tu nuevo repositorio.
5. Ve a la pestaña "Actions" en GitHub para ver el pipeline en ejecución.
6. Una vez finalizado, ve a tu perfil de GitHub, luego a la pestaña "Packages" para ver la imagen publicada.

5.5 Recursos y Referencias

- **Docker Security Best Practices:**

- <https://docs.docker.com/develop/security-best-practices/>

- **Docker Scan:**

- <https://docs.docker.com/engine/scan/>

- **GitHub Actions Documentation:**

- <https://docs.github.com/en/actions>

- **Build and push Docker images with GitHub Actions:**

- [Guía de Docker para GitHub Actions](#)

- **Introducción a Kubernetes:**

- <https://kubernetes.io/docs/concepts/overview/>

6 Anexos

6.1 Apéndice A: Resumen de Comandos de Docker y Docker Compose

Esta sección es una guía de referencia rápida para los comandos más utilizados en tu día a día con Docker.

6.1.1 Gestión de Imágenes (Docker)

Comando	Descripción
<code>docker images</code>	Lista todas las imágenes en tu máquina local.
<code>docker build -t <nombre>:<tag> .</code>	Construye una imagen a partir de un Dockerfile en el directorio actual.
<code>docker pull <imagen>:<tag></code>	Descarga una imagen desde un registro (por defecto, Docker Hub).
<code>docker rmi <imagen></code>	Elimina una o más imágenes locales.
<code>docker tag <origen> <destino></code>	Etiqueta una imagen existente con un nuevo nombre/tag.
<code>docker history <imagen></code>	Muestra las capas que componen una imagen y su historial.
<code>docker scan <imagen></code>	Analiza una imagen en busca de vulnerabilidades de seguridad conocidas.
<code>docker login</code>	Inicia sesión en un registro de contenedores (ej. Docker Hub).
<code>docker push <imagen></code>	Sube una imagen a un registro de contenedores.

6.1.2 Gestión de Contenedores (Docker)

Comando	Descripción
<code>docker run <imagen></code>	Crea y arranca un nuevo contenedor a partir de una imagen.
<code>docker ps</code>	Lista los contenedores que están actualmente en ejecución.
<code>docker ps -a</code>	Lista todos los contenedores, incluyendo los que están detenidos.
<code>docker start <contenedor></code>	Inicia un contenedor detenido.
<code>docker stop <contenedor></code>	Detiene un contenedor en ejecución de forma segura (envía SIGTERM).
<code>docker kill <contenedor></code>	Detiene un contenedor en ejecución de forma abrupta (envía SIGKILL).
<code>docker rm <contenedor></code>	Elimina uno o más contenedores detenidos.
<code>docker rm -f <contenedor></code>	Fuerza la eliminación de un contenedor, incluso si está en ejecución.
<code>docker logs <contenedor></code>	Muestra la salida (logs) de un contenedor.
<code>docker logs -f <contenedor></code>	Sigue la salida de los logs en tiempo real.
<code>docker exec -it <contenedor> <cmd></code>	Ejecuta un comando dentro de un contenedor en ejecución (ej. <code>bash</code>).
<code>docker inspect <contenedor></code>	Proporciona información detallada (configuración, red, etc.) sobre un contenedor.

6.1.3 Gestión del Sistema (Docker)

Comando	Descripción
<code>docker system prune</code>	Elimina todos los contenedores detenidos, redes no utilizadas, imágenes colgantes y cachés de build.
<code>docker system prune -a --volumes</code>	Limpieza agresiva: elimina todo lo de prune, además de todas las imágenes no utilizadas y todos los volúmenes. ¡Úsalo con cuidado!
<code>docker volume ls</code>	Lista todos los volúmenes.
<code>docker volume create <nombre></code>	Crea un nuevo volumen.
<code>docker volume rm <nombre></code>	Elimina uno o más volúmenes.
<code>docker network ls</code>	Lista todas las redes.
<code>docker network create <nombre></code>	Crea una nueva red.
<code>docker network rm <nombre></code>	Elimina una o más redes.
<code>docker info</code>	Muestra información detallada sobre la instalación de Docker.
<code>docker version</code>	Muestra la versión del cliente y del servidor de Docker.

6.1.4 Comandos de Docker Compose

Comando	Descripción
<code>docker-compose up</code>	Construye (si es necesario), crea y arranca todos los servicios definidos en <code>docker-compose.yml</code> .
<code>docker-compose up -d</code>	Ejecuta los servicios en segundo plano (detached mode).
<code>docker-compose up --build</code>	Fuerza la reconstrucción de las imágenes antes de iniciar los servicios.
<code>docker-compose down</code>	Detiene y elimina los contenedores y las redes creadas por up.
<code>docker-compose down -v</code>	Además de down, elimina los volúmenes nombrados definidos en el archivo.
<code>docker-compose ps</code>	Lista los contenedores de los servicios definidos.
<code>docker-compose logs</code>	Muestra los logs de todos los servicios.
<code>docker-compose logs -f <servicio></code>	Sigue los logs de un servicio específico en tiempo real.
<code>docker-compose build</code>	Construye o reconstruye las imágenes de los servicios.
<code>docker-compose start</code>	Inicia los servicios existentes (previamente creados/detenidos).
<code>docker-compose stop</code>	Detiene los servicios sin eliminarlos.
<code>docker-compose exec <servicio> <cmd></code>	Ejecuta un comando en un servicio en ejecución (ej. bash).
<code>docker-compose config</code>	Valida y visualiza el archivo Compose procesado.

6.2 Apéndice B: Solución de Problemas Comunes

Aquí tienes una lista de los problemas más frecuentes y cómo solucionarlos.

6.2.1 1. Error: permission denied while trying to connect to the Docker daemon socket

- **Problema:** Este error ocurre casi siempre en **Linux** cuando intentas ejecutar un comando `docker` sin ser el usuario `root` y sin haber configurado los permisos correctamente.
- **Solución:** Debes añadir tu usuario al grupo `docker`.
 1. Abre una terminal.
 2. Ejecuta `sudo usermod -aG docker $USER`.
 3. Cierra sesión y vuelve a iniciarla, o ejecuta `newgrp docker` en tu terminal para aplicar los cambios en la sesión actual.

6.2.2 2. Error: Bind for 0.0.0.0:8080 failed: port is already allocated

- **Problema:** Estás intentando mapear un puerto en tu máquina (-p 8080:80) que ya está siendo utilizado por otra aplicación (o por otro contenedor Docker).
 - **Solución:**
 1. **Opción A (Cambiar el puerto):** Elige un puerto diferente en tu máquina. Por ejemplo, si el 8080 está ocupado, usa el 8081: `docker run -p 8081:80 ...`
 2. **Opción B (Encontrar y detener el proceso):** Encuentra qué proceso está usando el puerto y deténlo si no es necesario. Puedes usar `sudo lsof -i -P -n | grep LISTEN` en macOS/Linux o `netstat -ano | findstr :8080` en Windows para encontrar el ID del proceso.
-

6.2.3 3. Un contenedor no puede conectarse a otro (ej. API a Base de Datos)

- **Problema:** Tu contenedor de aplicación no puede resolver el nombre de host del contenedor de la base de datos (error ECONNREFUSED o Host not found).
 - **Causa Común:** Los contenedores no están en la misma red de Docker. Por defecto, cada contenedor se ejecuta en su propia red aislada a menos que se especifique lo contrario.
 - **Solución:**
 1. Crea una red común: `docker network create mi-red`.
 2. Al ejecutar los contenedores, añádelos a esa red con el flag `--network`:
 - `docker run --name db --network mi-red ...`
 - `docker run --name api --network mi-red ...`
 3. Si usas **Docker Compose**, este problema se resuelve automáticamente, ya que Compose crea una red por defecto para todos los servicios del archivo. Asegúrate de usar el **nombre del servicio** (ej. db) como el hostname en tu cadena de conexión, no localhost ni 127.0.0.1.
-

6.2.4 4. Los cambios en mi código no se reflejan en el contenedor

- **Problema:** Has configurado un "bind mount" para hot-reloading, pero cuando modificas un archivo en tu editor, la aplicación dentro del contenedor no se actualiza.
- **Solución:**
 1. **Verifica la ruta del volumen:** Asegúrate de que la ruta en tu máquina local y la ruta dentro del contenedor en la sección volumes de tu `docker-compose.yml` son correctas. Un error tipográfico es la causa más común.
 - Ejemplo: `- ./mi-codigo:/app`
 2. **Asegúrate de que la herramienta de hot-reloading esté funcionando:** El "bind mount" solo sincroniza los archivos. Necesitas una herramienta como nodemon (para Node.js), uvicorn `--reload` (para Python), etc., que vigile los cambios de archivo y reinicie el servidor. Asegúrate de que esté instalada y sea parte del CMD de desarrollo.
 3. **Problemas con WSL 2 (Windows):** Si trabajas desde Windows con WSL 2, el rendimiento y la notificación de eventos de archivos son mucho mejores si tu código reside dentro del sistema de archivos de WSL 2 (ej. en `~/proyectos/`) en lugar de en el sistema de Windows (ej. `/mnt/c/Users/...`).

6.3 Apéndice C: Recursos Adicionales y Comunidad

El ecosistema de Docker es vasto y activo. Aquí tienes algunos recursos para seguir aprendiendo y resolver dudas.

6.3.1 Documentación y Guías Oficiales

- **Docker Docs:** La fuente de verdad. La documentación oficial es excelente, completa y tiene guías para casi todo.
 - <https://docs.docker.com/>
 - **Play with Docker:** Un laboratorio online y gratuito para experimentar con Docker en tu navegador sin instalar nada.
 - <https://labs.play-with-docker.com/>
 - **Docker Hub:** Explora miles de imágenes oficiales y de la comunidad. Leer los Dockerfile de las imágenes oficiales es una gran forma de aprender.
 - <https://hub.docker.com/>
-

6.3.2 Comunidad y Foros de Ayuda

- **Stack Overflow:** El principal foro para hacer preguntas técnicas. Usa las etiquetas docker y docker-compose. Antes de preguntar, busca si tu duda ya ha sido resuelta.
 - <https://stackoverflow.com/questions/tagged/docker>
 - **Foros de la Comunidad Docker:** Un espacio oficial para discutir y obtener ayuda de otros usuarios.
 - <https://forums.docker.com/>
 - **Reddit:** Los subreddits r/docker y r/devops son lugares muy activos para noticias, discusiones y ayuda.
 - <https://www.reddit.com/r/docker/>
 - <https://www.reddit.com/r/devops/>
-

6.3.3 Blogs y Tutoriales Recomendados

- **Blog oficial de Docker:** Para anuncios, nuevas características y casos de estudio.
 - <https://www.docker.com/blog/>
- **The Docker Captains Blog:** Publicaciones de expertos de la comunidad reconocidos por Docker. Contenido de alta calidad y muy práctico.
 - <https://www.docker.com/captains/>
- **DigitalOcean Community Tutorials:** Tienen una enorme cantidad de tutoriales prácticos y bien explicados sobre Docker y temas relacionados.
 - <https://www.digitalocean.com/community/tags/docker>